Closing the Gap: Photogrammetry-Based Simulation for Sim-to-Real Transfer of Reinforcement Learning Navigation on the Pepper Robot

Jaishil Patel - 2602867
School of Computer Science
University of the Witwatersrand
Johannesburg, South Africa

Pravesh Ranchod
School of Computer Science
University of the Witwatersrand
Johannesburg, South Africa

Branden Ingram
School of Computer Science
University of the Witwatersrand
Johannesburg, South Africa

***Abstract*—Reinforcement learning can produce navigation policies in simulation, but these policies often fail on real robots because the simulator differs from reality. This study asks whether building the simulator from a photogrammetric scan of the deployment room narrows this sim-to-real gap for the Pepper humanoid robot. A real room was scanned with a tablet to create a photogrammetric simulator, and a standard simulator of the same room was built by hand from tape measurements, so that the source of the geometry was the only difference between them. Depth-based navigation policies were trained in each with Soft Actor-Critic under identical domain randomisation and deployed without further training on the physical Pepper. The photogrammetry policy's success rate dropped by only 8 percentage points when measured by odometry-estimated success on the real robot, compared with a 37-point drop for the standard policy, showing that it transferred better from simulation to reality. It also achieved odometry-estimated success on 9 of 20 real routes against 6, although this difference was not significant. Failures occurred where the robot's built-in collision protection stopped it near obstacles. For the two policies tested, the results suggest that a reconstructed room yields a simulator whose results transfer more faithfully to the real robot, while modelling the robot's own behaviour remains necessary.**

***Index Terms*—Sim-to-Real, Photogrammetry, Reinforcement Learning, Soft Actor-Critic, Pepper Robot**

I. Introduction

Reinforcement learning (RL) lets a robot learn navigation by trial and error, but training on a real robot is slow, costly and unsafe [1], [2]. Policies are therefore trained in simulation, where they often fail once moved to the real robot because the simulator differs from reality. This is known as the sim-to-real gap [2], [3]. It can be narrowed in two ways: by making the simulator a closer copy of reality, or by making the policy robust to the differences that remain, for example with domain randomisation [3], [4].

This report tests the first approach on the Pepper robot, whose hardware is too fragile for training on the robot itself and for which no open simulator yet supports RL navigation [5]. A real room was scanned with a tablet and turned into a photogrammetric simulator. A second, standard simulator of the same room was built by hand from tape measurements. Navigation policies were trained with Soft Actor-Critic (SAC) under the same domain randomisation in each simulator and then deployed, without further training, on the physical Pepper over the same 20 routes. Because the source of the room's geometry is the only difference between the two, the comparison isolates the effect of the reconstruction.

Using odometry-estimated success on the real robot, the photogrammetry policy showed a smaller performance drop from simulation to reality than the standard policy, and it also reached the goal more often, although that difference was not statistically significant. Real-robot failures for both policies happened at the same two obstacles, where the robot's built-in collision protection stopped it.

Data and code availability. The code, simulator worlds, route sets and evaluation results are available at [TODO: https://github.com/...].

II. Research Question and Hypothesis

Problem. Earlier work has shown that grounding a simulator in the real scene can improve transfer [6], [7], but it has not isolated the contribution of the reconstruction itself.

Research question. Does training Pepper in a photogrammetry-based simulation of a room improve its real-world navigation and reduce the sim-to-real gap, compared with a standard simulation of the same room built by hand?

Hypothesis. Compared with a policy trained in the same way with SAC under domain randomisation in a standard simulation of the room, a policy trained in a photogrammetric reconstruction of that room will show (H1) a smaller sim-to-real gap and (H2) a higher real-world success rate.

III. Background

A. Reinforcement Learning and Soft Actor-Critic

Reinforcement learning (RL) models sequential decision-making as a Markov Decision Process (MDP): at each step the agent observes a state, takes an action, and receives a reward, and it learns a policy that maximises the expected discounted sum of future rewards [1]. In mobile-robot navigation, the state usually combines a depth image with the position of the goal relative to the robot, and the action is a continuous linear and angular velocity [8].

Soft Actor-Critic (SAC) [9] is the algorithm used in this study. Besides maximising reward, it rewards the policy for staying random by adding an entropy term to the objective:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\big[r(s_t,a_t) + \alpha\,\mathcal{H}(\pi(\cdot \mid s_t))\big], \qquad (1)$$

where $\mathcal{H}$ is the entropy of the policy and the temperature α sets how much exploration is valued relative to reward. A policy that does not commit too early to one way of solving the task is less likely to overfit to the simulator. SAC is off-policy: it stores past experience in a replay buffer and reuses it many times, learns two critics with slowly updated target copies to avoid overestimating values, and tunes α automatically [9]. It suits this study because Pepper's actions are continuous, because reusing experience matters when every simulated step runs in real time, and because it has already been used for sim-to-real navigation [8].

B. The Sim-to-Real Gap and Domain Randomisation

A policy trained in simulation often fails on the real robot, because the simulator is only an approximation of reality [2], [3]. Zhao et al. [2] divide this *reality gap* into mismatches in sensing, such as camera behaviour, and in actuation and dynamics, such as friction, mass and latency. This study uses *zero-shot transfer*: the policy runs on the real robot without any further training.

Domain randomisation varies the simulator's parameters during training, so that the range of conditions the policy sees is wide enough to include the real world [4]. *Dynamics randomisation* varies physical parameters such as friction and actuator delay [3], and *observation randomisation* varies what the sensors report, for example by adding noise or dropping readings. Too much randomisation can harm rather than help real-world performance [2], so this study sets the ranges from measurements of the real robot where possible (Section V).

C. Photogrammetry

Photogrammetry reconstructs a textured 3D model of a real space from many overlapping photographs, by matching the same visual features across images taken from different viewpoints and working out their 3D positions [10]. The resulting mesh can be imported into a robot simulator. Alhamadah et al. [10] show that consumer devices such as smartphones can produce such models with a mean dimensional error of about 5%.

D. The Pepper Robot and ROS 2

Pepper is a wheeled humanoid service robot made by SoftBank Robotics and widely used in research [5], [11]. Its hardware is too fragile for repeated real-world training, so simulation is needed [5]. Its depth camera has a narrow field of view, and its visual localisation drifts over distance and is sensitive to lighting [12], which is why this study uses depth images and a single room. The robot is connected to the policy through ROS 2 [13], whose named data channels (topics) are used in the same way in the Gazebo simulator [14] and on the real Pepper, so the policy's inputs and outputs stay identical across the transfer.

IV. Related Work

In their survey of sim-to-real methods, Zhao et al. [2] identify domain randomisation as the most widely used and note that navigation has received much less attention than manipulation. Broadly, earlier work narrows the gap from one of two directions: making the policy robust to the mismatch that remains, or making the simulator a more faithful copy of reality. This study combines both.

A. Making the Policy Robust

Tobin et al. [4] introduced domain randomisation by varying the visual appearance of simulated scenes, and Peng et al. [3] showed that randomising the dynamics of a simple simulator can work better than carefully building a detailed one. For navigation, Chaffre et al. [8] trained a depth-based policy with SAC and deployed it on a real wheeled robot, doubling the real-world success rate of a naive baseline to 47%. Their training environment, however, was a generic Gazebo world, not a model of the room the robot was deployed in.

B. Making the Simulator Realistic

Liu et al. [6] built a simplified simulation of a real scene from its semantic layout, trained a navigation policy in it and transferred it back, reaching 83% real-world success against 61% for plain domain randomisation and 15% for direct transfer. This shows that grounding the simulator in the real scene helps, although their reconstruction was an abstract layout rather than a realistic model. Photogrammetry can provide such a model: Guerra et al. [15] used it to render realistic camera images for drones, and note that it is still uncommon in robot simulation, where general-purpose simulators such as Gazebo [14] were not designed for realistic rendering. Habitat [16] trains navigation agents in 3D-scanned indoor scenes and evaluates them with the SPL metric of Anderson et al. [17], also used in this study. More recent work reconstructs real scenes with neural radiance fields or Gaussian splatting and trains policies inside them [7], [18], [19]. Closest to this study, Chhablani et al. [7] reconstruct scenes captured with a mobile phone, train navigation agents in them and report a strong correlation between simulated and real performance, although for a different robot and reconstruction method.

What these studies do not isolate is the contribution of the reconstruction itself. They compare a realistic simulator against domain randomisation or against no simulator, not against a simulator of the *same* room built by hand to the same measurements. This study makes that comparison.

C. Work on Pepper

Gómez et al. [12] built a visual navigation system for Pepper but reported drift and sensitivity to lighting. Sekkat et al. [5] found existing Pepper simulators to be of low fidelity, closed-source and tied to the outdated ROS 1, and built an open ROS 2 digital twin, leaving learning-based control as future work.

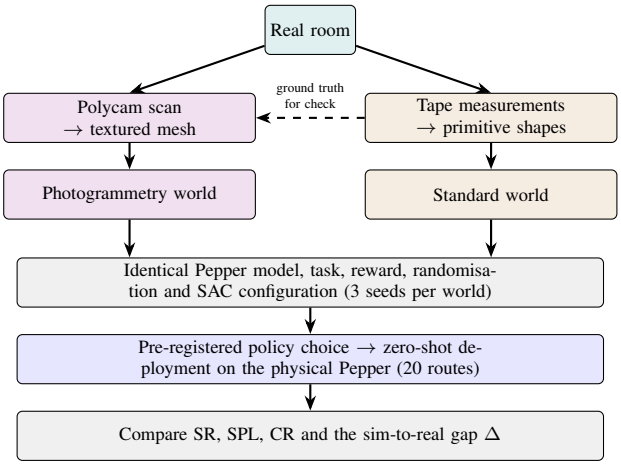


Fig. 1. The experimental pipeline. The two branches (purple: photogrammetry (method), brown: standard (baseline)) differ only in the source of the room geometry; every later stage is shared.

To our knowledge, no study has trained an RL navigation policy for Pepper in a photogrammetric reconstruction of its environment and tested whether the reconstruction improves transfer to the real robot.

V. Methodology

A. Overview

This section describes how the experiment was built and run. The design follows one rule: the two simulated environments may differ only in where their geometry comes from. Everything else, including the robot model, physics settings, task, reward, randomisation, learning algorithm and route set, is shared. The photogrammetry world is the method under test; the standard world, built by hand from tape measurements, is the baseline it is compared against. Fig. 1 shows the full pipeline, which has five stages:

1) Capturing the real room with photogrammetry and checking the model against tape measurements.
2) Building two simulated versions of the room: the photogrammetric mesh, and a standard model made of simple shapes built from the same tape measurements.
3) Integrating Pepper into the simulator and calibrating it against the real robot.
4) Training navigation policies with SAC under domain randomisation in each world.
5) Deploying one policy from each world on the physical Pepper and comparing the results.

B. The Photogrammetry Branch

The target room is a 5.5×4.2 m storage room containing a couch, two chairs, a shelf, three large boxes, a bag, a green pole and a red pole that serves as the navigation goal (Fig. 3). It was captured with the Polycam application in photo mode on an iPad Pro 11-inch (2nd generation). Over a 10–30 minute session, overlapping photographs were taken while walking around the room, at several heights, and around each piece of furniture so that it was seen from all sides. The room lights were on, nothing was moved during the capture, and no people were in the room. Polycam then processed the photographs into a textured 3D mesh using photogrammetry (Section III), which was exported as a GLB file. The mesh was already at metric scale, so no scale correction was applied; the accuracy gate below confirms this.

A custom import script (`mesh_import.py`) converted the mesh into a simulator model in a repeatable way. It aligned the room with the simulator axes, placed the origin at the floor corner, smoothed scanning noise with a light Taubin filter, and reduced the mesh to 50,000 faces (using Blender) so the physics engine could run it in real time. Every transform was saved to a manifest file so the import can be reproduced exactly.

Before training, the model had to pass an accuracy gate. Ten room dimensions ($K = 10$) were measured with a tape measure, including room length, width and height and the heights of the furniture, and the same dimensions were measured on the mesh. The mean absolute percentage error was

$$\bar{e} = \frac{1}{K} \sum_{k=1}^{K} \frac{|m_k - \hat{m}_k|}{m_k} \times 100\%, \tag{2}$$

where m_k is the tape measurement and $\hat{m}_k$ the mesh measurement. The threshold was the 4.97% reported for consumer-grade photogrammetry by Alhamadah et al. [10]. The reconstruction achieved $\bar{e} = 2.12\%$. The largest errors were on soft objects (couch height 6.85%, bag height 5.70%), and the room dimensions were within about 1%.

Two problems appeared once Pepper drove on the scanned floor. First, the floor was tilted by $1.78°$, so the mesh was levelled to $0.01°$. Second, small bumps in the scanned floor caught Pepper's wheels and left the robot stuck. To fix this, only the *collision* mesh was flattened below 0.12 m (high enough to remove the bumps in the scanned floor, but lower than any real object in the room). The visual mesh, which the depth camera sees, was not changed, and the accuracy gate still gave 2.12%.

C. The Standard Branch

The standard world is the baseline an engineer would build by hand. It was generated from the same tape measurements as the gate: walls, a flat floor, and one box-shaped primitive per furniture item, each with its measured size and position. What it lacks is the irregular surface detail that photogrammetry recovers, such as the shape of the couch, the open shelf compartments and the bag. Both world files are generated from a single template, and an automatic check confirms that the only difference between them is which room model is included. Fig. 2 compares the two worlds.

D. Pepper in Simulation

Pepper was simulated in Ignition Gazebo (Fortress) with ROS 2 Humble, using the DART physics engine with Bullet collision detection and a 5 ms step, and an open-source Pepper

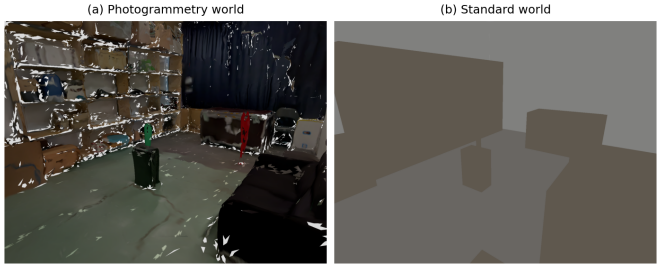


Fig. 2. The same view of the photogrammetry world (left) and the standard world (right). The white flecks are texture artefacts from mesh decimation; they affect colour only, and the policy observes depth.

description that was ported to ROS 2 Humble for this project. The policy only uses the same ROS 2 topics that exist on the real robot: the depth image from the head camera, the odometry of the base, and velocity commands to the base. This means the same code drives both the simulated and the physical robot. The head was fixed at a pitch of 0.45 rad (looking down) in both simulation and reality. At Pepper's default upward head angle, low obstacles such as the bag and the boxes leave the depth image when they are still about 3 m away.

Collisions are detected with contact sensors on eight links (base, torso, upper arms, forearms and hands), and a tilt of more than 20° is also treated as a collision (normal driving never tilts Pepper by more than about 6°, so a larger tilt means the robot is falling over). On the real robot, the three base bumpers provide the same collision signal.

The simulated base was then calibrated against the real robot. The same set of velocity commands (straight, reverse, spins and arcs) was sent to both, and their responses were compared. The real robot reached its commanded speeds to within about 5%, but responded with a median delay of 0.85 s through the full communication chain, while the simulated base responded almost instantly. The simulator was therefore given a four-step (0.8 s) command delay and acceleration limits fitted to the real measurements.

E. The Navigation Task

The task is point-to-point navigation. Pepper starts at a pose in the room and must reach the red goal pole, at $(3.75, 1.10)$ m, without colliding. It is modelled as an MDP as follows.

Observation. At each step the policy receives:

- a 64×48 depth image, limited to 0.4–8.0 m (the range of the simulated camera; the real camera's readings were cut to the same range, and anything outside it, including missing readings, is treated as far away);
- the goal relative to the robot, as $(d, \sin\theta, \cos\theta)$, where d is the distance and θ the bearing;
- the last six actions. Because commands take effect about 0.8 s after they are sent, this history tells the policy what it has already asked the robot to do.

Action. A continuous linear velocity $v \in [-0.10, 0.30]$ m/s and angular velocity $\omega \in [-0.90, 0.90]$ rad/s, sent at 5 Hz.

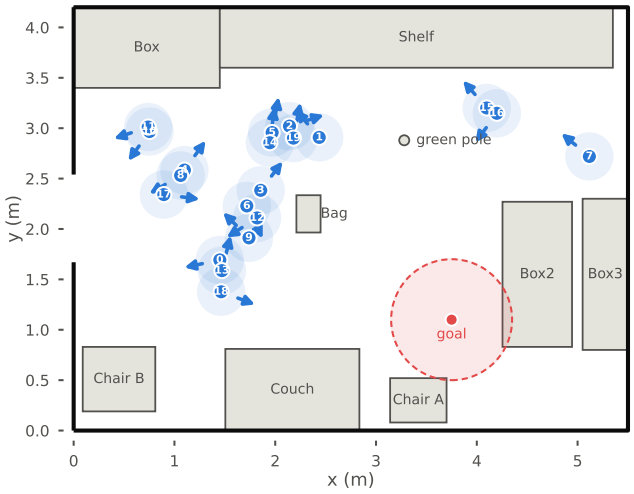


Fig. 3. Top-down plan of the room with the 20 validation route starts (blue, arrows show the starting heading) and the 0.6 m goal zone around the red pole.

Reward. Each step's reward is

$$r_t = -0.05 + 20\,(d_{t-1} - d_t) - 0.02\,|\omega_t|, \tag{3}$$

which consists of a small time penalty, a reward for every metre of progress towards the goal, and a small penalty for turning. In addition, $+100$ is given for reaching the goal and -100 for a collision.

Episode end. An episode ends when Pepper's base centre is within 0.6 m of the goal (success), when it collides or tips over, or after 120 s.

Starts and routes. During training, each episode starts at a random collision-free position with a random heading, so that the policy cannot memorise the validation starts. For evaluation, a fixed set of 20 validation routes was generated (Fig. 3). The shortest feasible path length ℓ^* of each route was planned on the tape-measured map with obstacles enlarged by 0.32 m to allow for Pepper's body. The same 20 routes are used for both branches, in simulation and on the real robot. They were also used to choose which seed to deploy, so they are validation routes rather than an untouched test set. Before the final training runs, short development runs on them led to corrections of the simulation: the scanned floor was levelled, contact sensors were added, the camera was tilted down to see low obstacles, and the goal tolerance and tip-over rule were set. The action history and the latency, depth-noise and frame-rate randomisation were added or set from measurements of the real robot rather than from route results. The reward, network and SAC hyperparameters were set before the final training runs and not changed after. The evaluations every 25,000 steps were used only for monitoring; the deployed policies are the final ones at 300,000 steps.

F. Domain Randomisation

Both branches were trained with the same randomisation (Table I). Each parameter is sampled at the start of every

TABLE I
RANDOMISATION APPLIED DURING TRAINING, IDENTICAL FOR BOTH BRANCHES

Group	Parameter	Range
Command	Linear / angular velocity scale	$\times$0.8–1.2
	Linear velocity noise (std)	0.02 m/s
	Angular velocity noise (std)	0.05 rad/s
	Command delay (around 4 steps)	$\pm$2 steps
	Wheel slip probability	0–0.05
Depth	Per-pixel noise (std)	0.01 m
	Pixel and patch dropout	0–15%
	Frame rate (log-uniform)	1.5–15 Hz
Surface	Floor friction coefficient μ	0.6–1.2

episode and held fixed for that episode, following Peng et al. [3]. Where possible, the ranges were set from measurements of the real robot rather than chosen by hand. The command delay is varied around the measured 0.8 s. The depth noise matches the 5.5 mm median frame-to-frame noise measured on the real camera. The dropout patches imitate a full-height band with no depth readings that a window produced in the real camera's image. The depth frame rate is varied because the real camera delivered only 1.9–3 frames per second over Wi-Fi during calibration, well below the 5 Hz control rate. When no new frame arrives, the policy receives the previous one again.

G. Training with Soft Actor-Critic

The policies were trained with the SAC implementation in Stable-Baselines3 [9], [20]. A small convolutional network, the NatureCNN encoder from DQN [21], encodes the depth image into 128 features, which are joined with the goal vector and action history and passed to two fully connected layers of 256 units. The hyperparameters were the same for every run:

- learning rate 3×10^{-4}, discount $\gamma = 0.99$, target update $\tau = 0.005$;
- replay buffer of 100,000 transitions, batch size 256, 2,000 warm-up steps;
- one gradient step per environment step, and automatic entropy tuning;
- 300,000 environment steps per run.

Three runs (seeds 0, 1 and 2) were trained per branch, six in total. One run takes 20–33 hours of wall-clock time, so the runs were executed in parallel on the Wits mathematical sciences cluster inside an Apptainer container. Training saved checkpoints every 25,000 steps and resumed automatically when the simulator stalled. Every 25,000 steps, each run was evaluated once on the 20 routes.

After training, each final policy was evaluated in its own world over five independent sessions of the 20 routes. Several sessions are needed because the simulator runs in real time rather than in fixed steps, so repeated runs of the same policy do not give identical results. Before any real-robot run, a rule was fixed for choosing which policy from each branch to deploy: the seed with the highest simulated SR, with ties broken by SPL. This selected standard seed 1 and photogrammetry seed 0.

H. Deployment on the Real Pepper

The policies were deployed zero-shot, with no fine-tuning on the real robot. Pepper's software (NAOqi 2.5) only supports Python 2, so a small Python 2 program on the laptop talks to the robot over the network, and a ROS 2 bridge node republishes its data under the same topic names as the simulator. The policy therefore runs unchanged. The depth image (160×120 requested at 5 Hz), odometry, bumper states and velocity commands all pass through this link. As a safety measure, the robot stops if no command arrives for 1.5 s, and the emergency stop button was always within reach. NAOqi's built-in collision protection, which slows or stops the base near obstacles, could not be disabled and stayed on for all real trials.

The 20 routes were taped onto the floor of the real room. For each route, the operator placed Pepper on the taped start, and its odometry was reset to that known pose. The policy then ran until success, collision or 120 s. Contacts that the base bumpers cannot detect, such as an arm brushing furniture, were noted by the operator. The evaluation took place over two days (routes r000–r010 and r011–r019), and the order of the two policies was swapped on the second day.

I. Evaluation Metrics

Four metrics are used. Over the $N = 20$ routes, the *success rate* is $\mathrm{SR} = \frac{1}{N}\sum_i S_i$, where $S_i = 1$ if route i ended within 0.6 m of the goal with no collision before the time limit, and 0 otherwise. On the real robot, no external tracking was available, so whether a route succeeded and the distance travelled both come from odometry, reset at the taped start of each route. We call this odometry-estimated success. Real-robot SR and SPL are therefore odometry-estimated, and the sim-to-real gap compares them with simulated values computed from the true pose.

The *success weighted by path length* [17] also rewards efficient paths:

$$\mathrm{SPL} = \frac{1}{N}\sum_{i=1}^{N} S_i \, \frac{\ell_i^*}{\max(\ell_i, \ell_i^*)}, \tag{4}$$

where ℓ_i is the distance actually travelled. The *collision rate* CR is the number of collisions per metre travelled. The *sim-to-real gap* for a metric M is

$$\Delta M = M_{\mathrm{sim}} - M_{\mathrm{real}}, \tag{5}$$

computed separately for each branch. A smaller Δ means the simulator predicted real performance better.

The main comparison is between branches: photogrammetry is considered better if Δ_{photo} is smaller than Δ_{std}. This is tested per route with the Wilcoxon signed-rank test ($\alpha = 0.05$) on the paired differences $\Delta_{\mathrm{photo},i} - \Delta_{\mathrm{std},i}$, where a route's gap is its mean simulated outcome over the five sessions minus its real outcome. Real-robot performance of the two policies is also compared directly. Both tests pair each route with itself,

TABLE II

FINAL SIMULATED PERFORMANCE (MEAN ± STANDARD DEVIATION OVER FIVE SESSIONS OF 20 ROUTES). * MARKS THE POLICY DEPLOYED ON THE REAL ROBOT.

Branch	Seed	SR	SPL	CR (per m)
Standard	0	0.67±0.06	0.54±0.05	0.09
	1*	0.67±0.08	0.59±0.06	0.17
	2	0.39±0.07	0.37±0.06	0.46
	Mean	0.58	0.50	0.24
Photogrammetry	0*	0.53±0.08	0.48±0.07	0.30
	1	0.44±0.15	0.38±0.12	0.36
	2	0.29±0.09	0.27±0.08	0.59
	Mean	0.42	0.38	0.42

TABLE III

THE TWO DEPLOYED POLICIES IN SIMULATION AND ON THE REAL ROBOT (20 ROUTES EACH). REAL VALUES ARE ODOMETRY-ESTIMATED. Δ IS SIMULATED MINUS REAL; p IS THE PAIRED PER-ROUTE WILCOXON TEST.

	Standard (seed 1)	Photogrammetry (seed 0)
Sim SR / SPL	0.67 / 0.59	0.53 / 0.48
Real SR	0.30 [0.15, 0.52]	0.45 [0.26, 0.66]
Real SPL	0.16	0.27
Real CR (per m)	0.032	0.055
ΔSR (p)	0.37 (0.009)	0.08 (0.392)
ΔSPL (p)	0.43 (0.001)	0.21 (0.022)
Is $\Delta_{\text{photo}} < \Delta_{\text{std}}$? SR: $p = 0.03$, SPL: $p = 0.01$		

because both branches use the same 20 routes. The same paired test checks whether each branch's own gap is larger than zero. 95% confidence intervals for the real-robot SR use the Wilson score interval.

VI. RESULTS AND DISCUSSION

This section first reports how the policies performed in simulation and on the real robot. It then compares the sim-to-real gap of the two branches, which is the main question of this study, and finally looks at why the real robot failed on most routes.

A. Simulation Results

Table II shows the final simulated performance of all six policies, each evaluated over five sessions of the 20 routes. The standard policies performed better in simulation, with a mean SR of 0.58 against 0.42 for photogrammetry. The seeds also differed a lot: standard seed 2 reached only 0.39, while seeds 0 and 1 both reached 0.67. A likely reason for the lower photogrammetry scores is that its world is harder to navigate: the scanned obstacles have irregular shapes, and the mesh has small cracks that leave gaps in the depth image (Fig. 2; appendix, Fig. 7), whereas the standard world contains only clean boxes. The training curves are given in the appendix (Fig. 5). Following the rule fixed before the real trials, standard seed 1 and photogrammetry seed 0 were deployed on the robot.

B. Real-Robot Results

Both deployed policies performed worse on the real robot than in simulation (Table III). By odometry-estimated success, the photogrammetry policy succeeded on 9 of 20 routes (SR 0.45) and the standard policy on 6 of 20 (SR 0.30). The photogrammetry policy therefore did better on the real robot, but the 95% confidence intervals overlap, and the paired per-route comparison does not show a significant difference (SR $p = 0.18$, SPL $p = 0.07$).

Collisions were rarer on the real robot than in simulation (0.032 and 0.055 per metre). This is mainly because NAOqi's collision protection, which could not be turned off, slowed or stopped the base near obstacles. Instead of colliding, the robot often stopped in front of an obstacle until the time ran out (Section VI-D).

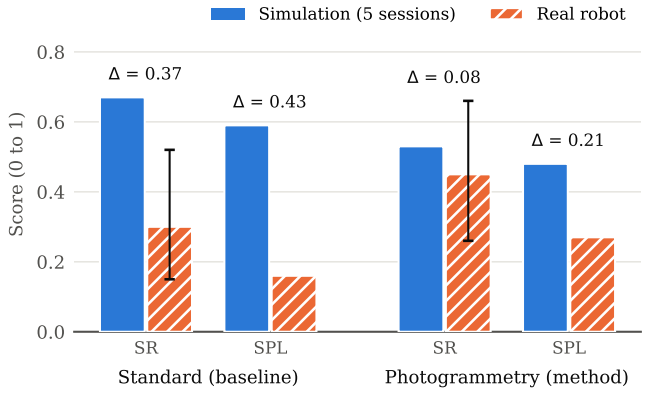


Fig. 4. SR and SPL of the two deployed policies in simulation and on the real robot. Error bars on the real SR are 95% Wilson intervals; Δ is the sim-to-real gap.

C. The Sim-to-Real Gap

The photogrammetry policy lost far less performance when moved to the real robot (Fig. 4). Its success rate fell by only 8 percentage points (from 0.53 in simulation to 0.45 on the robot), while the standard policy's fell by 37 points (from 0.67 to 0.30). Measured by SPL, which also rewards short paths, the photogrammetry policy's drop (0.21) was about half that of the standard policy (0.43). Comparing the two gaps route by route, the photogrammetry gap was smaller on 12 routes and the standard gap on 3, and the difference is statistically significant ($p = 0.03$ for SR, $p = 0.01$ for SPL). This supports the hypothesis for the two deployed policies: the policy trained in the photogrammetry world transferred with a smaller loss of performance. Because the test pairs routes rather than independent training runs, it does not show that this holds across training seeds (Section VII).

The smaller gap has two sources. The photogrammetry policy scored lower in simulation (0.53 against 0.67), so its simulator gave a less optimistic prediction, and it scored higher on the real robot (0.45 against 0.30). The standard policy's gap was large and significant (SR $p = 0.009$, SPL $p = 0.001$), so its simulator clearly over-predicted real performance. For the photogrammetry policy, only the SPL gap was significant ($p = 0.022$), while its SR gap was not ($p = 0.392$): its

TABLE IV
HOW EACH REAL ROUTE ENDED (20 ROUTES PER POLICY)

Outcome	Standard	Photogrammetry
Reached the goal	6	9
Collided (all with the couch)	2	3
Stopped at the couch or bag until timeout	5	5
Stopped within 1 m of the goal (near miss)	2	0
Barely moved (< 1 m)	1	3
Other timeout	4	0

simulated success rate was a reasonable prediction of its real one, although it reached the goal by less efficient paths on the real robot.

D. Why the Real Robot Failed

Table IV groups every real route by how it ended, using the scored result and the operator's notes. Two factors stand out.

The couch and the bag. Many failures happened at two obstacles. The couch lies between many of the starts and the goal, and the bag stands in the middle of the room (Fig. 3). Every collision was with the couch, and many more routes ended with the robot stopped in front of the couch or the bag. In simulation, the policies reach the goal from the left of the room by driving through the gap between the bag and the couch, passing close to the couch (appendix, Fig. 6). On the real robot this same gap is where most runs failed: NAOqi's collision protection held the base back when it came close to an obstacle, while the policy kept commanding it towards the goal. The policy had never experienced this behaviour in training, so it had no way to back off and try another path.

Real-robot conditions. The real robot also differed from the simulator in ways the randomisation only approximated. The control loop ran at 5 Hz over Wi-Fi with depth images at 160×120, and commands took effect about 0.85 s after they were sent. Together with the collision protection, these conditions made the robot slower and more hesitant than in simulation, which helps explain the many timeouts and the low SPL on the real robot.

VII. LIMITATIONS

The study has several limitations that affect how far its results can be generalised:

- **Scale of the evaluation.** The study used one room, 20 routes and one real-robot session per policy, and only one trained policy from each branch was deployed. The samples are small, the confidence intervals on the real success rates are wide, and the results may not hold in other rooms. The 20 routes pair the two policies route by route, but they are not independent training runs, so the p-values describe these two selected policies on these routes. They do not show that photogrammetry-trained policies in general transfer better across training seeds.
- **Seed selection.** The deployed policy in each branch was the seed with the highest simulated SR on the same validation routes. Choosing the best of three seeds on these routes biases its simulated SR upwards, and therefore inflates Δ, for both branches.
- **Position measurement.** No external tracking system was available. The robot's final position came from only the odometry estimates.
- **Robot conditions.** NAOqi's collision protection could not be turned off and stopped the robot near obstacles, which the policies never experienced in training. The robot was controlled over Wi-Fi at 5 Hz with a command delay of about 0.85 s, and mass and centre of mass were not randomised.
- **Simulation fidelity.** The simulator runs in real time and is not deterministic, so repeated evaluations of the same policy differ and five sessions were needed. The scanned mesh has small cracks, and its largest errors were on soft objects (up to 6.85%). The room was static, with no people or moving objects. Depth readings closer than 0.4 m are treated as far away, although, with the camera 1.1 m above the floor, this only affects objects already touching the robot.

VIII. FUTURE WORK

Several extensions would strengthen and extend these results:

- **Robustness test.** A natural next step is a perturbed version of the room, with the bag and the green pole moved and ten new routes. It would show whether the photogrammetry policy also copes better when the room no longer matches its scan.
- **Position tracking and more trials.** An overhead camera or motion-capture system would remove the dependence only on odometry. More real sessions, and more than one deployed policy per branch, would narrow the confidence intervals.
- **Collision protection.** Many real failures happened when NAOqi's collision protection held the robot in front of the couch or the bag. Modelling this behaviour in simulation, or training with it, would remove the largest source of failure.
- **Other rooms and reconstructions.** Repeating the comparison in several rooms, and with newer reconstruction methods such as Gaussian splatting [7], would show how general the benefit of a reconstructed simulator is.

IX. CONCLUSION

This study built two simulators of the same room for Pepper, one from a photogrammetric scan and one by hand from tape measurements, trained a SAC navigation policy in each under identical domain randomisation, and deployed both on the real robot without further training.

On H1, the photogrammetry policy had a significantly smaller sim-to-real gap: its success rate fell by 8 percentage points from simulation to the real robot, against 37 for the standard policy. On H2, its odometry-estimated success was 9 of 20 real routes against 6, which points the same way but is not statistically significant. Most real failures of both policies

happened at the couch and the bag, where collision protection, unseen in training, stopped the robot.

For the two policies tested, the results suggest that a reconstructed room transfers to the real robot more faithfully than a hand-built one made from the same measurements. Whether this holds across training seeds needs more than one deployed policy per branch. Photogrammetry alone is not enough, however: transfer also depends on modelling the robot's own behaviour, such as its collision protection, and confirming the real-world difference needs larger trials with external position tracking.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[2] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," in *Proc. IEEE Symp. Series on Computational Intelligence (SSCI)*, 2020, pp. 737–744.

[3] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2018.

[4] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2017.

[5] H. Sekkat, O. Moutik, B. El Kari, Y. Chaibi, T. Ait Tchakoucht, and A. El Hilali Alaoui, "Beyond simulation: Unlocking the frontiers of humanoid robot capability and intelligence with Pepper's open-source digital twin," *Heliyon*, vol. 10, no. 7, p. e34456, 2024.

[6] N. Liu, Y. Cai, T. Lu, R. Wang, and S. Wang, "Real–sim–real transfer for real-world robot control policy learning with deep reinforcement learning," *Applied Sciences*, vol. 10, no. 5, p. 1555, 2020.

[7] G. Chhablani, X. Ye, M. Z. Irshad, and Z. Kira, "EmbodiedSplat: Personalized real-to-sim-to-real navigation with Gaussian splats from a mobile device," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2025, pp. 25 431–25 441.

[8] T. Chaffre, J. Moras, A. Chan-Hon-Tong, and J. Marzat, "Sim-to-real transfer with incremental environment complexity for reinforcement learning of depth-based robot navigation," in *Proc. 17th Int. Conf. Informatics in Control, Automation and Robotics (ICINCO)*, 2020.

[9] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Machine Learning (ICML)*, 2018.

[10] A. Alhamadah, M. Mamun, H. Harms, M. Redondo, Y.-Z. Lin, J. Pacheco, S. Salehi, and P. Satam, "Photogrammetry for digital twinning industry 4.0 (I4) systems," arXiv:2407.18951, 2024.

[11] A. K. Pandey and R. Gelin, "A mass-produced sociable humanoid robot: Pepper: The first machine of its kind," *IEEE Robotics & Automation Magazine*, vol. 25, no. 3, pp. 40–48, 2018.

[12] C. Gómez, M. Mattamala, T. Resink, and J. Ruiz-del Solar, "Visual SLAM-based localization and navigation for service robots: The Pepper case," in *RoboCup 2018: Robot World Cup XXII*, 2018.

[13] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[14] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, vol. 3, 2004, pp. 2149–2154.

[15] W. Guerra, E. Tal, V. Murali, G. Ryou, and S. Karaman, "FlightGoggles: Photorealistic sensor simulation for perception-driven robotics using photogrammetry and virtual reality," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2019, pp. 6941–6948.

[16] M. Savva, A. Kadian, O. Maksymets *et al.*, "Habitat: A platform for embodied AI research," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019.

[17] P. Anderson, A. Chang, D. S. Chaplot *et al.*, "On evaluation of embodied navigation agents," arXiv:1807.06757, 2018.

[18] A. Byravan, J. Humplik, L. Hasenclever, A. Brussee, F. Nori, T. Haarnoja, B. Moran, S. Bohez, F. Sadeghi, B. Vujatovic *et al.*, "NeRF2Real: Sim2real transfer of vision-guided bipedal motion skills using neural radiance fields," arXiv:2210.04932, 2022.

[19] M.-Y. Shen, C.-C. Hsu, H.-Y. Hou, Y.-C. Huang, W.-F. Sun, C.-C. Chang, Y.-L. Liu, and C.-Y. Lee, "DriveEnv-NeRF: Exploration of a NeRF-based autonomous driving environment for real-world performance validation," arXiv:2403.15791, 2024.

[20] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[21] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

APPENDIX

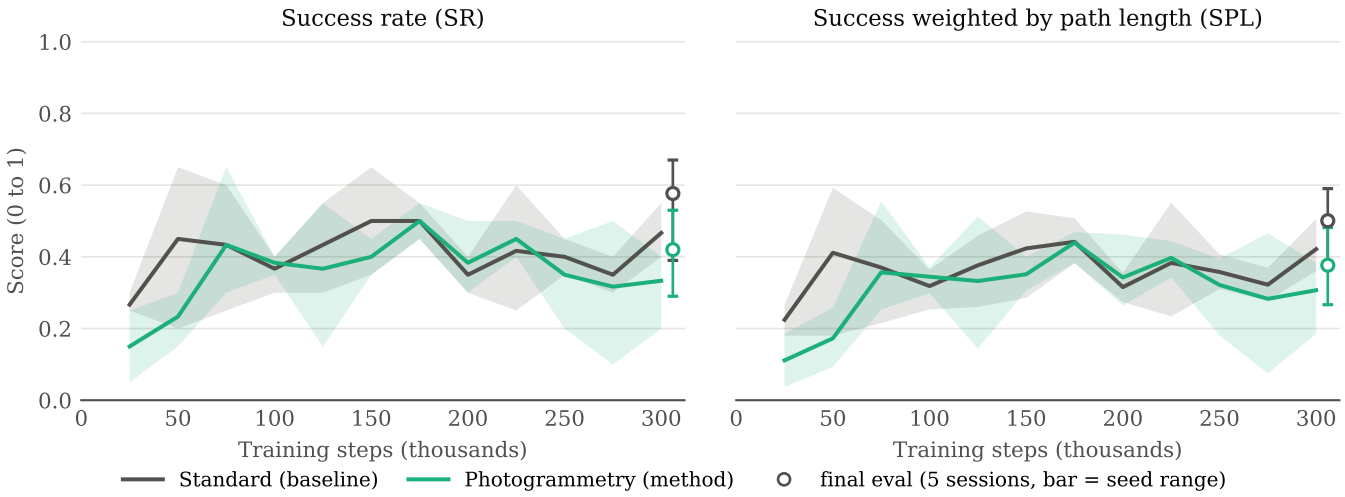


Fig. 5. Simulated SR (left) and SPL (right) during training, measured on the 20 routes every 25,000 steps (one session per point). Lines are the mean of the three seeds of each branch and the shaded bands the range across seeds. Hollow markers at 300k are the final five-session evaluations used in Table II (seed mean, bar showing the seed range). Single-session evaluations of one seed jump by as much as 0.5 between neighbouring points, which is why the final results use five sessions.

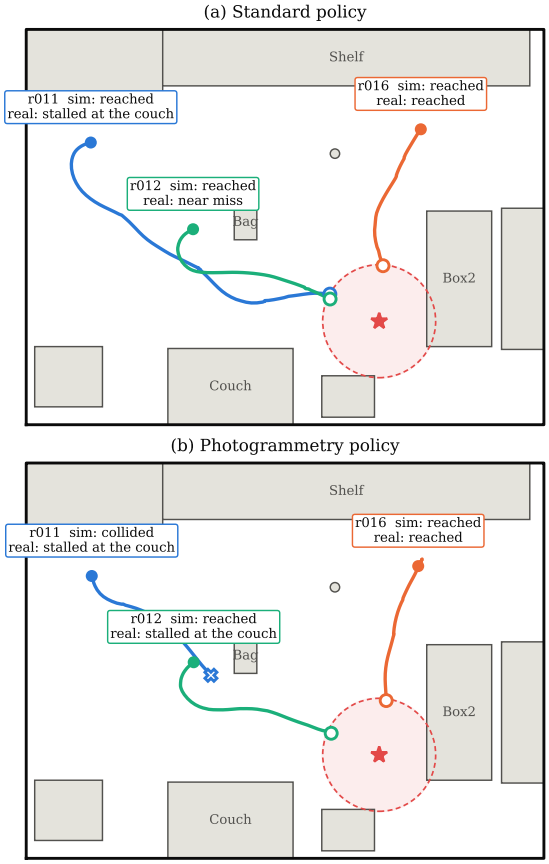


Fig. 6. Simulated paths of the two deployed policies on three example routes (one rollout each; filled dot: start, hollow circle: reached, cross: collision). The label gives the outcome of the same route in simulation and on the real robot, whose path was not recorded.

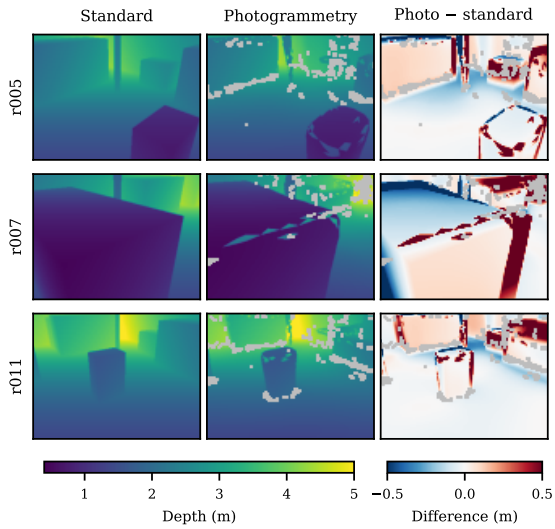


Fig. 7. Simulated depth images from identical poses in the two worlds: the head camera at the start of three routes, turned to face the goal, with no robot in the scene. The right column is photogrammetry minus standard depth. Grey pixels have no depth reading and are read by the policy as far away; in the photogrammetry world they mostly come from small cracks in the scanned mesh. The two worlds agree on the walls and floor, and differ mainly at the edges of objects, where the scanned shapes replace the standard world's boxes.

Wits University Faculty of Science post-graduate student AI declaration

I understand that the use of generative AI tools (such as ChatGPT or similar) without explicitly declaring such use constitutes a form of plagiarism and is classified by Wits University as academic misconduct.

I declare that in the course of conducting the research towards my degree or in the preparation of this thesis/dissertation/research report (select one by marking with an X):

I **did not** make use of generative AI tools ☐

I **did** make use of generative AI tools for the following (tick all that apply):

1. Idea Generation (research problem/design, hypothesis) ☐
2. Sourcing Related Work (summarising, identifying sources) ☐
3. Methods and Experiment Design (experiment setup, model tuning) ☑
4. Data Analysis (presentation, coding, interpretation) ☑
5. Theoretical Development (theorem proving, conceptual analysis) ☐
6. Code Development (generating algorithms, writing scripts) ☑
7. Presentation (rendering graphics, formatting) ☐
8. Editing (grammar, readability) ☑
9. Writing (text generation, document structuring) ☐
10. Citation Formatting (structuring, organising) ☐

If other uses were involved, please specify below:

Generative AI tool used (list all)	**Used for?**

If generative AI tools were used as an integral part of the experimental design or in the direct execution of my research, I confirm that details of this use are clearly outlined in the relevant experimental/methodology chapters of my thesis/dissertation/research report.

Student number: 2602867

Candidate signature: 

Date: 22/10/2026